

Práctica con Listas en Python

En este documento encontrarás una serie de actividades para practicar el uso de Listas y Tuplas en Python. ¡Vamos a programar! 🐍💻

1 Actividad - Traductor de frases 🧠

📌 Consigna:

Tenemos cinco frases en inglés y necesitamos un programa que las traduzca a los idiomas más utilizados.

🧩 Pasos a seguir:

1. Importar la función `Translator` del módulo `translate`.
2. Crear una **lista con las siglas** de los idiomas soportados.
3. Pedirle al usuario que ingrese la **sigla del idioma** al que quiere traducir las frases.
 - Pasar lo que ingrese a **minúsculas** con `.lower()`.
4. Crear un `while` que verifique si la sigla está dentro de la lista.
 - Si no lo está, seguir pidiendo la sigla hasta que sea válida.
5. Crear una lista con las **5 frases en inglés**:

```
frases = [  
    "Well done is better than well said",  
    "Never look back",  
    "You only fail when you stop trying",  
    "The best dreams happen when you are awake",  
    "The power of imagination makes us infinite"  
]
```

6. Crear una **lista vacía** llamada `frasesTraducidas` para guardar las frases ya traducidas.

7. Crear una función `traducir_frases(idioma)` que reciba la sigla del idioma deseado y:
 - Itere con un `for` sobre cada frase.
 - Utilice `translator.translate(frase)` y agregue la traducción a la lista `frasesTraducidas`.
 - Retorne esa lista traducida.
8. Mostrar el mensaje:

Las frases traducidas son:

y luego imprimir la lista resultante.

2 Actividad - Meses del año

Consigna:

Desarrollar un programa que permita cargar los siguientes valores en una lista: "ENERO", "MARZO", "MAYO", "JUNIO", "SEPTIEMBRE", "NOVIEMBRE"; y luego permita:

- Completar la lista con los meses faltantes y mostrar la lista completa. Cada mes faltante debe ser agregado en la posición correcta: es decir, al mostrarlos deben aparecer como:
"ENERO", "FEBRERO", "MARZO", "ABRIL", "MAYO", "JUNIO", "JULIO", "AGOSTO", "SEPTIEMBRE", "OCTUBRE", "NOVIEMBRE", "DICIEMBRE"
- Quitar de la lista los meses que comienzan con la letra "M" o "N". Antes de finalizar mostrar la lista final en memoria de la siguiente manera:
"ENERO", "FEBRERO", "ABRIL", "JUNIO", "JULIO", "AGOSTO", "SEPTIEMBRE", "OCTUBRE", "DICIEMBRE"

Nota: Utilice un ciclo para recorrer la lista al mostrarla, (puedes usar un ciclo `for` iterable si lo deseas).

3 Actividad 3: Lista de números

Consigna:

Desarrollar un programa que permita al usuario cargar números enteros en una lista. Luego de ingresar cada valor se debe mostrar la lista con los valores ingresados hasta el momento (el proceso de carga debe terminar cuando el usuario ingresa 0 como número de entrada).

Luego de cargar la lista realizar lo siguiente:

- Solicitar un número entero n y modificar la lista mediante el borrado de todos los elementos que sean iguales a este valor.
- Ordenar la lista de mayor a menor.

Nota: Implementar la solución utilizando funciones.

4 Actividad 4: Procesa palabras

Consigna:

Diseñar un programa que solicite al usuario una cantidad n de palabras a cargar en una lista y luego permita realizar las siguientes tareas por medio de funciones:

- Cargar una lista llamada words[] con n palabras.
- Mostrar la lista cargada.
- Solicitar al usuario una palabra a buscar en words[] y determinar cuántas veces aparece en la lista.
- Solicitar dos palabras y sustituir la primera por la segunda en la lista words[].
- Solicitar una palabra y eliminarla de la lista words[].
- Crear una segunda lista igual a la primera, pero donde sus elementos estén dispuestos al revés, es decir, en orden inverso (no se trata de escribir la lista al revés, sino de crear una lista distinta).

5 Actividad 5: Estaciones del año

Consigna:

Diseñar un programa que a partir de una lista con los meses del año, cree las siguientes sub-listas:

- Lista `mesesVerano[]`, debe incluir los meses para los cuales la estación del año es verano en el hemisferio sur.
- Lista `mesesOtonio[]`, debe incluir los meses para los cuales la estación del año otoño en el hemisferio sur.
- Lista `mesesInvierno[]`, debe incluir los meses para los cuales la estación del año es invierno en el hemisferio sur.
- Lista `mesesPrimavera[]`, debe incluir los meses para los cuales la estación del año es primavera en el hemisferio sur

6 Actividad 6: Procesa números

Consigna:

Diseñar un programa que permita cargar una lista de números enteros positivos llamada `positivos[]` y a través del uso de funciones realice las siguientes tareas:

- Función `primos()`: permite devolver una nueva lista con todos los número primos de la lista `positivos[]`.
- Función `operaciones()`: permite calcular la sumatoria y el promedio de los valores de la lista `positivos[]` y mostrar los resultados en pantalla.
- Función `tresListas()`: permite solicitar un número `n` y generar tres nuevas listas a partir de la lista `positivos()`: una con los números mayores a `n`, otra con los número menores a `n` y otra con los números iguales a `n`.

Nota: La carga de la lista `positivos[]` finalizará cuando el usuario ingrese -1 como valor de entrada.

7 Actividad 7: Año académico de estudiante

Consigna:

Diseñar un algoritmo que solicite los datos de un alumno de la UNdeC en cuanto a nro. de legajo, carrera y año de cursado y luego permita realizar las siguientes tareas:

- Cargar en una lista llamada `asignAI[]` las asignaturas cursadas por un alumno y luego mostrarla en pantalla.
- Cargar en otra lista llamada `promAs[]` el promedio obtenido por el alumno en cada asignatura y luego mostrarla en pantalla. Notar que ésta lista esta directamente relacionada con la lista `asigAI[]`.

- Eliminar de la listas anteriores las asignaturas regularizadas y cargarlas en una nueva lista anidada llamada `asignReg[]` que contiene como elemento otras listas de estructura `[asignatura, promedio]`. Tener en cuenta que para estar aprobada la nota del promedio de la asignatura debe ser mayor o igual a 6.

Nota: Antes de finalizar, el programa debe mostrar por pantalla las asignaturas regularizadas y aquellas en las cuales está libre.

8 Actividad 8: Final de asignatura

Consigna:

Desarrollar un programa que permita cargar una lista con las notas obtenidas por los alumnos en el examen final de una asignatura. El usuario ingresará las notas de los alumnos una a una para armar la lista en memoria. La carga finaliza cuando se ingrese como nota el valor -1. Una vez cargada en memoria la lista de notas desarrollar las siguientes tareas por medio de funciones:

- a) Función `mostrarNotas()`: permite mostrar en pantalla la lista de notas.
- b) Función `mayorNota()`: permite determinar la mayor nota y la muestra en pantalla.
- c) Función `menorNota()`: permite determinar la menor nota y la muestra en pantalla.
- d) Función `tresListas()`: permite solicitar una nota `n` y generar tres listas adicionales: una con las notas mayores a `n`; otra con las notas menores a `n`; y otra con las notas iguales a `n`.

9 Actividad 9: Notas de asignatura mejorado

Consigna:

Se solicita optimizar el programa anterior para permitir cargar el listado de alumnos que rindieron el final de una asignatura con los siguientes datos: matrícula, nombre, apellido y nota obtenida en el final. El programa se deberá realizar implementando un menú con las siguientes opciones:

1. Opción 1 - Cargar listado de alumnos.

Permite cargar el listado de alumnos que rindieron el final con los datos requeridos. La carga del listado finaliza cuando se ingresa 0 como matrícula. Mostrar el listado cargado antes de salir de la opción.

2. Opción 2 – Alumnos aprobados

Permite mostrar un mensaje en pantalla tal como ‘Final aprobado’ para todos aquellos alumnos del listado que hayan obtenido una nota superior o igual a 4.

3. Opción 3 – Alumnos desaprobados

Permite mostrar un mensaje en pantalla tal como ‘Final desaprobado’ para todos aquellos alumnos del listado que hayan obtenido una nota inferior a 4

4. Opción 4 – Salir

Nota: para la resolución del enunciado debe implementar listas anidadas.

10 Actividad 10: Biblioteca

 **Consigna:**

Diseñar un programa en Python que permita manejar el registro de libros de una biblioteca usando listas anidadas. Los datos requeridos para cada libro son los siguientes: código, título, autor, unidades disponibles y cantidad de veces que fue prestado. El programa deberá desarrollar la funcionalidad por medio de un menú con las siguientes opciones:

1. Opción 1 - Agregar un libro.
2. Opción 2 - Buscar un libro por código.
3. Opción 3 - Eliminar un libro.
4. Opción 4 - Modificar datos de un libro.
5. Opción 5 - Mostrar listado de libros.
6. Opción 6 - Mostrar el libro más leído.
7. Opción 7 - Mostrar el libro menos leído.
8. Opción 8 - Mostrar cantidad de préstamos realizados en total
9. Opción 9 - Salir

Nota: para la resolución del enunciado debe implementar listas anidadas.

Actividad Extra - Tuplas: Menú del día

 **Consigna:**

Un restaurante tiene un menú fijo del día compuesto por 3 platos: entrada, principal y postre. Cada plato está representado por una **tupla** que contiene:

1. El nombre del plato.
2. Las calorías aproximadas.

Pasos a seguir:

1. Crear tres tuplas:
 - Una para la entrada.
 - Una para el plato principal.
 - Una para el postre.
2. Mostrar el menú completo utilizando `print()` con los valores de cada tupla.
3. Sumar todas las calorías y mostrar por consola:

El menú completo tiene un total de ____ calorías.